National Research University Higher School of Economics
Faculty of Computer Science
Bachelor's Programme Data Science and Business Analytics
01.03.02 Applied Mathematics and Information Science

# Internship report

Fulfilled by

Shutov Alexey Andreevich
*(Surname, Given Name, Middle Name if any)*

*(signature)*

**Supervised by**

| | | |
|---|---|---|
| *Senior Lecturer: HSE Tikhonov Moscow Institute of Electronics and Mathematics / School of Applied Mathematics* | *Titova Natalia Nikolaevna* | |
| *(job or academic title)* | *(surname, initials)* | *(signature)* |


**Moscow, 2025**

**1 Introduction** **2**
1.1 Background 2
1.2 Problem Statement 2
1.3 Objectives 2
**2 Literature Review** **3**
**3 Datasets** **6**
**4 Metrics** **7**
**5 The choice of the solution methods** **9**
5.1 Recommender Net with Label Encoder 9
5.2 Label Encoder NCF (Neural Collaborative Filtering) 11
5.3 Content-Based Recommender System using BM25 and Ratings 12
5.4 Content-Based Filtering with TF-IDF 14
5.5 SASRec and SVD Hybrid Recommender 16
5.6 NCF using Microsoft Recommenders Library 18
5.7 LightGBM 19
5.8 Content-Based Filtering on Goodreads 20
**Conclusion** **21**
**Приложение** **23**

# 1 Introduction

## 1.1 Background

Online e-commerce platforms have boomed during the last 5 years. Everyone is watching Netflix, buying products from Amazon or Wildberries, watching videos on YouTube. All these have become a result of the personalized supply system of products to the customers. People get what they are interested in. Now it is common to call these systems as Recommender algorithms. They can take into various types of data depending on the result or aim. For example, 80% of movies on Netflix are watched as a result of work of the recommender systems[20]. Such systems play a significant role in overall sales of the e-commerce platforms.

Despite the fact that recommender systems have proved to be an effective instrument to boost the sales. They have some key factors which might limit their performance and effectiveness.

## 1.2 Problem Statement

Various recommender algorithms can have their limitations. The goal of the study is to conduct the research when a specific model should be used and what factors may affect its performance. In addition to that suggest improvements to be made. Give an insight into what data is the most appropriate for a chosen model.

## 1.3 Objectives

- Learn the fundamentals of the recommender systems
- Practice building the recommender models ML/DL
- Study the way of models' evaluation
- Get the intermediate results

- Compare the results of the students' work with the related researches.
- Study the ways to enhance the recommender models

# 2 Literature Review

Recommender systems are tools used to suggest items, products, or services to users based on various factors such as their preferences, past behavior, or characteristics of the items themselves. These systems are highly relevant in today's digital world, powering platforms like e-commerce websites, music streaming services, and social media networks. There are three primary types of recommendation approaches: Collaborative Filtering, Content-Based Filtering, and Hybrid Methods, as proposed by Adomavicius and Tuzhilin (2005)[1].

Collaborative filtering is a popular method that leverages the collective preferences of users to make recommendations. It comes in two forms: memory-based and model-based. Memory-based collaborative filtering involves computing similarities between users or items and suggesting items based on these similarities. In user-based collaborative filtering, recommendations are made by finding similar users and suggesting items they liked, as described by Sarwar et al. (2001)[2]. In item-based collaborative filtering, recommendations are based on items similar to those the user has interacted with, which is another approach highlighted by Sarwar et al. (2001)[2]. Model-based collaborative filtering, on the other hand, includes techniques like matrix factorization, where the system decomposes user-item interactions into lower-dimensional matrices, capturing hidden patterns between users and items, a method proposed by Koren et al. (2009)[3].

Content-based filtering makes recommendations based on the characteristics of the items and the preferences of the users. In this approach, items are described using features, such as genre, author, or keywords, and recommendations are

made by finding items similar to those the user has interacted with in the past. Lops, De Gemmis, and Semeraro (2011) discussed this in the context of content-based filtering methods for recommending items based on their inherent attributes[4]. Content-based methods help solve the cold start problem for new users but are limited in that they tend to recommend similar items, reducing diversity in suggestions, as noted by Lops et al. (2011)[4].

Hybrid methods combine collaborative filtering and content-based filtering techniques to overcome the shortcomings of each individual approach. A weighted hybrid method would combine the scores from both models, while a switching hybrid approach might switch between models depending on the context or the availability of data. For example, Netflix uses hybrid methods, incorporating both collaborative and content-based strategies to provide diverse recommendations, as described by Gomez-Uribe and Hunt (2016)[5].

Machine learning approaches to recommender systems involve using algorithms that can learn patterns from data. Supervised learning models, such as classification and regression techniques, are used to predict user preferences based on labeled data. These models learn from historical user behavior to predict ratings or the likelihood of interaction with an item. Unsupervised learning, including clustering techniques like k-means, groups users or items with similar characteristics, enabling recommendations based on these groupings, as described by Agarwal et al. (2018)[6]. More advanced models include reinforcement learning, which optimizes recommendations by learning from user interactions in a dynamic, reward-based environment, as proposed by Liang et al. (2018)[7].

Deep learning approaches in recommender systems have gained popularity in recent years due to their ability to model complex patterns. Deep neural networks, such as autoencoders, are used for collaborative filtering, where they

learn compressed representations of users and items. Neural collaborative filtering (NCF) is an example of a model where a neural network learns the interaction between users and items, enhancing the predictive accuracy of the recommendations, as proposed by He et al. (2017)[8]. Recurrent neural networks (RNNs) are used for sequential recommendations, as they capture temporal patterns in user behavior, such as recommending content based on past interactions over time, as demonstrated by Hochreiter and Schmidhuber (1997)[9].

Evaluating recommender systems involves measuring how well the system's predictions align with actual user behavior. Common accuracy-based metrics include root mean square error (RMSE) and mean absolute error (MAE), which measure the difference between predicted and actual ratings. Ranking-based metrics such as precision, recall, and F1-score are used to evaluate top-N recommendations, where a system suggests a list of items to the user. These metrics are widely discussed by Serrano et al. (2014)[10]. Additional metrics like hit rate, coverage, and diversity assess how representative or diverse the recommendations are, while business impact metrics focus on conversion rates and user engagement.

Despite their success, recommender systems face several challenges. The cold start problem arises when there is insufficient data for new users or items, making it hard to generate accurate recommendations. Scalability is another issue, as the number of users and items grows, making computations increasingly complex. Additionally, recommender systems often suffer from a lack of diversity, as they tend to recommend similar items over time, which can reduce the novelty of suggestions. Bias and fairness in recommendations is also a growing concern, as algorithms can unintentionally reinforce existing biases, favoring certain users or groups over others, as discussed by Ekstrand et al. (2016)[11].

In conclusion, the field of recommender systems has evolved significantly, from traditional collaborative filtering to more sophisticated machine learning and deep learning approaches. Each method has its strengths and weaknesses, and hybrid approaches aim to balance the trade-offs. While challenges remain, the future of recommender systems lies in refining existing models, making them more adaptive, fair, and explainable, while integrating new data sources and algorithms for more personalized and effective recommendations, as envisioned by Zhao et al. (2019)[12].

# 3 Datasets

To develop and evaluate the recommender system, a diverse set of datasets from different domains was utilized, including movies, local business reviews, and books. This variety allowed for comprehensive testing of the system's performance across varying types of user-item interactions and content structures.

The MovieLens datasets, a standard benchmark in recommender systems research, were employed in two versions: MovieLens 1M and MovieLens 100K. The MovieLens 1M dataset contains approximately one million user ratings and includes four key attributes: user ID, item ID, rating, and timestamp. Similarly, the MovieLens 100K dataset provides 100,000 ratings with additional metadata. In addition to user ID, item ID, rating, and timestamp, it includes the title and genres associated with each movie, as well as user-generated tags that serve as short comments. These datasets provide both explicit user feedback and auxiliary content information, making them well-suited for collaborative and content-based recommendation techniques.

In addition to movie-related data, business review data from Google Maps was used, specifically reviews of business locations in South Carolina. Two variations of this dataset were included. The first is a full review dataset containing user ID, the unique identifier for the business location (gmap_id), star rating, review text, and the time of the review. The second version is a simplified dataset which consists only of user ID, gmap_id, rating, and timestamp, focusing only on structured interactions without textual content. This data supports the modeling of user preferences in a location-based recommendation scenario.

Lastly, a dataset from Goodreads was incorporated to study the book recommendation use cases. This dataset consists of user ID, book ID, an indicator for whether the book was read(action), the rating provided by the user, incomplete review text (serving as a form of user comment), and the date the user started reading the book. This dataset provides a rich basis for testing hybrid recommendation models by combining elements of explicit feedback with behavioral metadata and user comments.

Together, these datasets provide a multifaceted basis for training, validating, and benchmarking the proposed recommender system across various domains and data structures.

# 4 Metrics

To assess the performance of the recommendation models, a combination of regression-based and ranking-based metrics was used. For models predicting explicit ratings: Root Mean Squared Error (RMSE) and Mean Squared Error (MSE) were utilized to evaluate the accuracy of numerical predictions. RMSE

provides an interpretable measure of the average deviation between predicted and actual ratings, while MSE emphasizes larger errors due to its squared nature, offering insights into the variance of prediction inaccuracies.

Additionally, Mean Absolute Percentage Error (MAPE) was employed to measure the average absolute error as a percentage of actual values, which is particularly useful when comparing errors across datasets with different rating scales.

For evaluating the quality of top-N recommendations, several ranking-based metrics were considered. Precision@K measures the proportion of relevant items among the top-K recommended items, providing an indication of recommendation accuracy. Recall@K evaluates the proportion of all relevant items that appear in the top-K recommendations, focusing on completeness. Normalized Discounted Cumulative Gain (NDCG@K) accounts for both the relevance and position of items in the recommendation list, rewarding methods that rank highly relevant items higher. Hit Rate@K (or Hit@K) was also used, measuring the proportion of users for whom at least one relevant item appears in the top-K recommendations. Together, these metrics offer a comprehensive evaluation framework, capturing both the predictive accuracy and the effectiveness of the models in generating relevant and well-ranked recommendations.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

$$\text{MAPE} = \frac{1}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right| \times 100$$

$$\text{NDCG@K} = \frac{1}{Z_K} \sum_{i=1}^{K} \frac{2^{rel_i} - 1}{\log_2(i+1)}$$

- $Z_k$ normalising factor ensuring that the value is between 0 - 1
- $rel_i$ relevance score of the item at position i
- $K$ number of top recommendations considered

$$\text{Precision@K} = \frac{\text{Number of relevant items in top-K recommendations}}{K}$$

$$\text{Recall@K} = \frac{\text{Number of relevant items in top-K recommendations}}{\text{Total number of relevant items}}$$

$$\text{Hit@K} = \frac{\text{Number of users with at least one relevant item in top-K recommendations}}{n_{\text{users}}}$$

# 5 The choice of the solution methods

Deep learning methods combined with machine learning were conducted.

## 5.1 Recommender Net with Label Encoder

Applied on the MovieLens 100K dataset using user and item IDs encoded with label encoders, incorporating additional metadata such as movie titles, genres, ratings, timestamps, and user-generated tags to model user preferences.

Recommender Net uses the MLP net under the hood and the textual data is encoded using Label Encoder (item_id and user_id) and the data like genres, tags etc was converted using MultilabelBinarizer

**RMSE - 0.68**

The same approach was conducted on Google Maps dataset but with the additional data as reviews and the normalized timestamps from 0 to 1 indicating 0 - beginning, 1 - end.

The raw text from each review was tokenized using Hugging Face's tokenizers library and converted into padded sequences of token IDs. The primary motivation was to capture content-based signals that can improve the model's ability to distinguish between similarly rated items. While no pre-trained embeddings or sentiment models were used, the inclusion of raw token IDs through an embedding layer allowed the model to learn basic textual patterns. This helped mitigate cold-start effects and sparsity issues commonly faced by recommender systems, as supported in prior work on text-aware recommendation models [13].

A Multi-Layer Perceptron (MLP) was used as the core model. The network took the following concatenated inputs:

- ➢ User embedding
- ➢ Item embedding
- ➢ Text representation (from token embeddings)
- ➢ Normalized time

The concatenated vector was passed through two dense layers with ReLU activation and dropout for regularization. The output layer produced a single scalar value representing the predicted rating. This approach was chosen for its flexibility in integrating heterogeneous input types and simplicity of implementation.

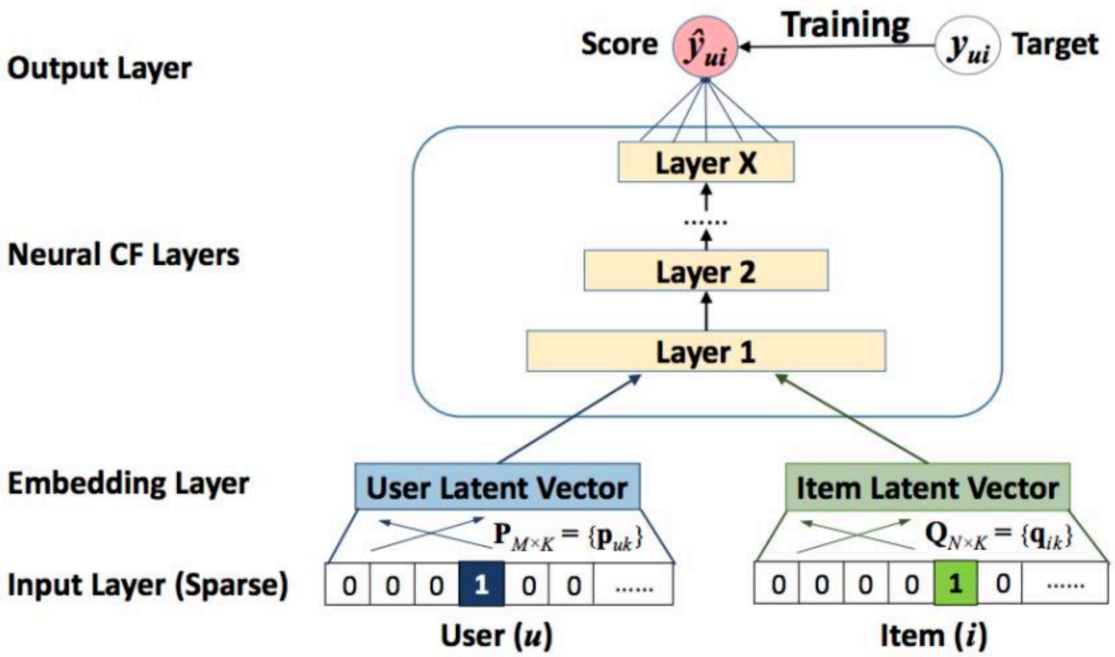
Training Procedure

Data was split chronologically: older reviews were used for training, while newer reviews were reserved for testing. This decision was based on the need to evaluate the model's ability to generalize to future user behavior, and to avoid temporal data leakage, which is a known issue in offline evaluation [14].

The model was trained using Mean Squared Error (MSE) loss with the Adam optimizer. Early stopping was used to avoid overfitting, and no explicit hyperparameter tuning was performed beyond embedding size and dropout rate. The obtained metric MAE was the best compared to the performance of other models. It showed 0.5825 accuracy which means that the given user model predicts the rating from 1 to 5 for a business location with the error of 0.58 points.

It means that adding more context as reviews improves the model's quality.

## 5.2 Label Encoder NCF (Neural Collaborative Filtering)



The Multi Layer Perceptron is the main element of the NCF.
Implemented on the MovieLens 100K dataset using user and movie identifiers with label encoding, trained on rating and timestamp data to learn latent user-item interactions.
Through the gathering of latent features via the neural network, NCF may identify complicated, nonlinear patterns in user-item interactions, resulting in enhanced recommendation efficacy in e-commerce contexts. Employing ReLU as the activation function for the MLP component facilitates the modeling of nonlinear patterns.
**MAPE = 29%**

## 5.3 Content-Based Recommender System using BM25 and Ratings

This experiment uses content-based recommendation techniques to generate movie recommendations based on metadata (genres and tags) using the BM25 algorithm, enhanced with movie ratings. The MovieLens dataset is utilized, incorporating movies, ratings, and tags to create a recommendation system that utilizes both content-based filtering and rating-based boosting.

The preprocessing steps involved:

1. Merging Tags with Movies: The tags dataset was aggregated by movieId, joining all the tags for a given movie into a single string. These were then merged with the movies dataset.
2. Combining Metadata: The genres and tags were concatenated into a single string, referred to as metadata, for each movie. The genres were separated by spaces, and missing tags were handled by filling NaN with an empty string.

3. Tokenization: The metadata was tokenized using the word_tokenize function from the Natural Language Toolkit (NLTK). The tokenized metadata serves as the input for the BM25 algorithm, which uses the tokenized text for information retrieval, as described by Robertson and Zaragoza (2009).

BM25

The BM25 algorithm is a probabilistic information retrieval method that ranks documents (in this case, movies) based on the query's relevance to the document's contents[15]. The BM25 score for each movie is computed based on its metadata, and the scores are then adjusted by incorporating the average rating of each movie.

- BM25 Scoring: For a given movie, the BM25 scores are computed based on the tokenized metadata. Each movie's metadata is compared to a query (which is the metadata of a target movie) to evaluate relevance.
- Rating Boost: To incorporate user preferences, the BM25 score for each movie is enhanced by a rating boost. The rating_weight is a parameter that adjusts the impact of ratings on the recommendation score. The rating boost is computed as a fraction of the movie's average rating divided by the maximum rating value (5 in this case).

Recommendation Function

The recommendation function, content_based_recommendations, accepts a movie ID and generates the top-N movie recommendations based on the BM25 score and the average rating boost. The steps are as follows:

1. Compute BM25 scores for the target movie.

2. Enhance the BM25 scores by adding the rating boost for each movie.
3. Sort the movies by the enhanced score and return the top-N recommendations, excluding the target movie itself.

To evaluate the recommendation system, the Normalized Discounted Cumulative Gain (NDCG) at 10 is computed.

Results

The evaluation yielded an **NDCG score of approximately 18%** (or 0.17) at top-10, which suggests that the content-based recommendation system has a moderate ability to recommend relevant movies but still requires improvement, especially in terms of ranking quality.

## 5.4 Content-Based Filtering with TF-IDF

This experiment applies a content-based recommendation approach to the MovieLens dataset, utilizing TF-IDF (Term Frequency-Inverse Document Frequency) vectorization to analyze the metadata of movies (genres and tags) and combining this with movie ratings. The system recommends movies based on their textual similarity to a target movie, with the added influence of average user ratings.

The dataset consists of the following:

- Movies dataset: Contains movie information, such as movieId, title, and genres.
- Tags dataset: Contains user-generated tags associated with movies, identified by movieId and tag.
- Ratings dataset: Contains ratings provided by users for movies, identified by userId, movieId, and rating.

The preprocessing steps include:

1. Merging Tags with Movies: The tags for each movie are aggregated by movieId, and the tags for a given movie are combined into a single string. This is merged with the movie's dataset.
2. Combining Metadata: The genres and tags for each movie are concatenated into a single metadata string. Genres are separated by spaces, and missing tags are handled by filling NaN with an empty string.
3. TF-IDF Vectorization: The metadata for each movie is converted into a numerical format using TF-IDF vectorization. TF-IDF measures the importance of words in each movie's metadata relative to the corpus of all movies [16].

TF-IDF vectorization is applied to the movie metadata to create a sparse matrix, where each movie's metadata is represented as a vector of features (terms). These vectors are then compared using cosine similarity, a common metric to evaluate the similarity between two non-zero vectors [17]. The cosine similarity score ranges from 0 to 1, where 1 indicates perfect similarity and 0 indicates no similarity.

Recommendations are generated for a given movie by:

1. Cosine Similarity: For a target movie (identified by its movieId), cosine similarity is computed between its metadata vector and those of all other movies in the dataset.
2. Rating Boost: To personalize recommendations based on user preferences, the similarity scores are adjusted by incorporating the movie's average rating. A weight factor (rating_weight) is used to scale the influence of the rating.

The rating is normalized to the range [0, 1] by dividing the average rating by 5 (the maximum rating).

3. Top-N Recommendations: The movies are sorted by their enhanced similarity score, and the top-N most similar movies (excluding the target movie itself) are selected as recommendations.

The obtained **NDCG@10 is 16%** which is moderate, but there is a field for the future enhancements.

## 5.5 SASRec and SVD Hybrid Recommender

Overview of the Matrix Factorization Approach with SVD

In this experiment, a matrix factorization approach is applied to the MovieLens dataset using Singular Value Decomposition (SVD). The goal is to predict missing ratings for movies by decomposing the user-item interaction matrix into latent factors. The recommendations are then evaluated using standard recommendation metrics such as Hit@K, NDCG@K, Precision@K, and Recall@K.

The dataset consists of the information about movies, including movieId and title.

The preprocessing steps involved:

1. Rating Normalization: The ratings in the dataset are normalized by dividing by the maximum rating (5 in this case). This converts the ratings into a 0-1 scale.
2. User-Item Matrix: A user-item interaction matrix is built using pandas' pivot function, where rows represent users, columns represent movies,

and values represent ratings. Missing values (unrated movies) are filled with zeros.

Singular Value Decomposition (SVD)

SVD is a matrix factorization technique used to decompose the user-item interaction matrix into three matrices: user factors, item factors, and a diagonal matrix of singular values. In this case, Truncated SVD is used, which reduces the dimensionality by keeping only the top-k singular values, thereby retaining the most significant latent factors.

Recommendation Generation

After performing the decomposition, recommendations for a specific user are generated:

1. User Ratings: The ratings for a target user (user 1 in this case) are extracted from the user-item matrix.
2. Reconstructed Scores: The scores for each movie are predicted by computing the dot product between the user's latent factors and the item factors.
3. Filtering Seen Movies: Movies that the user has already rated are excluded from the recommendations.
4. Top-N Recommendations: The top-N recommendations are selected by sorting the predicted scores in descending order and filtering out the movies already rated by the user.

The obtained results are as follows

Hit@10 ==== 0.15
NDCG@10 ==== 0.120666
Precision@10 ==== 0.015

Recall@10 ==== 0.15

## 5.6 NCF using Microsoft Recommenders Library

This study applies the Neural Collaborative Filtering (NCF) model for recommender systems, using the MovieLens 1M dataset. The model was built on the basis of the Microsoft open source library called recommenders.
The goal of the experiment is to assess the effectiveness of the NCF model, specifically the NeuMF (Neural Matrix Factorization) variant, in predicting user-item interactions and generating top-K recommendations.

The NCF model used in this study is based on the "NeuMF" architecture, a hybrid of matrix factorization and multi-layer perceptron (MLP) (He et al., 2017).

The performance of the model was evaluated using four common ranking-based metrics: MAP@K, NDCG@K, Precision@K, and Recall@K. These metrics were computed for K=10 to assess the top-K recommendation quality.

Results

The evaluation yielded the following results:

- MAP@10 ==== 0.0558
- NDCG@10 ==== 0.1354
- Precision@10 ==== 0.1248
- Recall@10 ==== 0.0728

These results indicate that while the model provides reasonable recommendations, there is room for improvement, particularly in terms of

relevance and recall. Although, these results are pretty much comparable with the results of the Microsoft's research.

## 5.7 LightGBM

LightGBM for Regression

LightGBM is a gradient boosting framework using tree-based learners, developed by Ke et al. (2017) with a focus on high efficiency and scalability for big data. It offers faster training speed and lower memory use than traditional boosting methods (experiments report up to a 20× speed-up with almost the same accuracy), making it a powerful regression model in the pipeline that can capture complex patterns while keeping training times reasonable.

NLTK and VADER for Sentiment Analysis

NLTK's VADER (Valence Aware Dictionary and Sentiment Reasoner) is a lexicon-based, rule-based sentiment analysis tool specifically attuned to social media text. It was introduced by Hutto and Gilbert (2014) and provides sentiment polarity scores (negative, neutral, positive, and a compound overall score) without requiring labeled training data, allowing the pipeline to quickly incorporate the emotional tone of text as a feature for downstream modeling.

Feature Engineering

Feature engineering time-related variables (hour of day, weekday, month, time-of-day segment, weekend indicator, and days since review) is used to capture temporal cycles and recency effects from timestamp data. Converting raw datetime information into these features allows the model to learn daily, weekly, or seasonal patterns (for example, higher user activity on weekends or

changes as reviews age), thereby improving predictive accuracy in time-dependent data.

**RMSE == 0.97**

## 5.8 Content-Based Filtering on Goodreads

Sentence Embedding using *all-mpnet-base-v2*

We use the pre-trained *all-mpnet-base-v2* model from SentenceTransformers to encode each book's textual information (e.g. description) into a 768-dimensional sentence embedding. This model is based on the MPNet transformer architecture and has been fine-tuned to produce semantically rich sentence representations, meaning that books with similar content have embeddings that are close in the vector space. By using such a state-of-the-art sentence embedding model, the system can capture contextual and semantic similarities between books' content, which improves the quality of content-based recommendations [18].

User Profiles via Median of Book Embeddings

To represent each user's preferences, we construct a user profile vector by taking the component-wise median of the embedding vectors of all books the user has interacted with or liked. This median-embedding approach yields a central representation of the user's taste in the same semantic space as the book embeddings, and is analogous to common content-based recommendation techniques that aggregate item feature vectors (e.g. via averaging) to form user profiles. The median provides a robust summary of the user's interests by reducing the influence of outlier items, and this profile vector can be directly

compared to candidate book vectors to find recommendations (Lops et al., 2011).

### Cosine Similarity for Recommendations

Our system computes cosine similarity between the user profile vector and each candidate book's embedding (or between item vectors for item-item similarity) to identify and rank relevant book recommendations. Cosine similarity measures the cosine of the angle between two vectors, making it a scale-invariant way to assess how closely the user's preference vector aligns with an item's vector in the high-dimensional embedding space. This similarity metric is widely used in information retrieval and recommender systems for comparing text or content embeddings, as it effectively highlights semantic resemblance while ignoring differences in vector magnitude [19].

# Conclusion

The experiments clearly show that including several data sources—textual reviews, metadata, timestamps, user behavior—improves the accuracy and relevance of recommendation systems. Deep learning methods—especially those leveraging MLPs and embeddings—shown success in capturing complex user-item interactions. Though interpretable, content-based techniques such as BM25 and TF-IDF showed only slight performance; semantic embeddings and rating-based boosts will greatly enhance their performance. Particularly those including user-generated content and temporal patterns, hybrid and

context-aware models show the most potential for creating scalable and customized recommendation solutions overall.


1) **Adomavicius, G., & Tuzhilin, A. (2005).** Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*.
2) **Sarwar, B., et al. (2001).** Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th international conference on World Wide Web*.
3) **Koren, Y., et al. (2009).** Matrix factorization techniques for recommender systems. *Computer*.
4) **Lops, P., De Gemmis, M., & Semeraro, G. (2011).** Content-based filtering. *Recommender Systems Handbook*.
5) **Gomez-Uribe, C. A., & Hunt, N. (2016).** The Netflix recommender system: Algorithms, business value, and innovation. *ACM Transactions on Management Information Systems*.
6) **Agarwal, D., et al. (2018).** Unsupervised machine learning for large-scale recommender systems. *Proceedings of the 12th International Conference on Web Search and Data Mining*.
7) **Liang, X., et al. (2018).** A survey of deep learning methods for recommender systems. *Proceedings of the 27th International Joint Conference on Artificial Intelligence*.
8) **He, X., et al. (2017).** Neural collaborative filtering. *Proceedings of the 26th International Conference on World Wide Web*.
9) **Hochreiter, S., & Schmidhuber, J. (1997).** Long short-term memory. *Neural Computation*.
10) **Serrano, J. D., et al. (2014).** Evaluation of recommender systems. *Journal of Computer Science & Technology*.
11) **Ekstrand, M. D., et al. (2016).** Bias in recommender systems. *Proceedings of the 10th ACM Conference on Recommender Systems*.
12) **Zhao, P., et al. (2019).** The future of recommender systems: Trends and challenges. *IEEE Transactions on Knowledge and Data Engineering*.
13) **Zheng, L., Noroozi, V., & Yu, P. S. (2017).** Joint deep modeling of users and items using reviews for recommendation. *Proceedings of the Tenth ACM International Conference on Web Search and Data Mining (WSDM)*.
14) **Meng, C., Zhang, H., Zhang, Y., et al. (2020).** Temporal Bias in Offline Evaluation for Recommender Systems. *arXiv preprint arXiv:2003.12696*.

15) **Robertson, S., & Zaragoza, H. (2009).** The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4), 333–389.
16) **Salton, G., & Buckley, C. (1988).** Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
17) **Baeza-Yates, R., & Ribeiro-Neto, B. (1999).** *Modern Information Retrieval*. Addison-Wesley.
18) **Reimers, N., & Gurevych, I. (2019).** Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *EMNLP-IJCNLP*.
19) **Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001).** Item-based collaborative filtering recommendation algorithms. *Proceedings of the 10th International Conference on World Wide Web*.
20) **Jalal, S., & Alturki, R. (2024).** *Enhancing User Experience through Machine Learning-Based Personalized Recommendation Systems: Behavior Data-Driven UI Design.* ResearchGate.

# Приложение

https://github.com/alexey077/diplom